Experiment No. 3

Perform text cleaning, perform lemmatization (any method), remove stop words (any method), label encoding. Create representations using TF-IDF. Save outputs.

Install & Import liabraries

```
import nltk
import pandas as pd
import re
import string

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer

# Download datasets
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package punkt to C:\Users\Siddharth
[nltk_data]     Raut\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to C:\Users\Siddharth
[nltk_data]     Raut\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package punkt_tab to C:\Users\Siddharth
[nltk_data]     Raut\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to C:\Users\Siddharth
[nltk_data]     Raut\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to C:\Users\Siddharth
[nltk_data]     Raut\AppData\Roaming\nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
True
```

```
data = {
    "Text": [
        "I love NLP, Deep Learning, and Machine Learning also!",
        "This is a very bad product.",
        "Natural Language Processing is a mind blowing technology.",
        "I hate spam emails and calls."
    ],
```

```
    "Label": ["Positive", "Negative", "Positive", "Negative"]
}

df = pd.DataFrame(data)
print(df)
```

```
                                                Text     Label
0  I love NLP, Deep Learning, and Machine Learnin...  Positive
1                        This is a very bad product.  Negative
2  Natural Language Processing is a mind blowing ...  Positive
3                      I hate spam emails and calls.  Negative
```

Text Cleaning

```
def clean_text(text):
    text = text.lower()  # Lowercase
    text = re.sub(r'\d+', '', text)  # Remove numbers
    text = text.translate(str.maketrans('', '', string.punctuation))  # Remo
    return text

df["Cleaned_Text"] = df["Text"].apply(clean_text)
```

Remove Stopwords + Lemmatization

```
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words]
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    return " ".join(tokens)

df["Processed_Text"] = df["Cleaned_Text"].apply(preprocess)

print(df[["Text", "Processed_Text"]])
```

```
                                                Text  \
0  I love NLP, Deep Learning, and Machine Learnin...
1                        This is a very bad product.
2  Natural Language Processing is a mind blowing ...
3                      I hate spam emails and calls.

                                     Processed_Text
0       love nlp deep learning machine learning also
1                                        bad product
2  natural language processing mind blowing techn...
3                               hate spam email call
```

Label Encoding

```
encoder = LabelEncoder()
df["Encoded_Label"] = encoder.fit_transform(df["Label"])

print(df[["Label", "Encoded_Label"]])
```

```
      Label  Encoded_Label
0  Positive              1
1  Negative              0
2  Positive              1
3  Negative              0
```

TF-IDF Representation

```
vectorizer = TfidfVectorizer()

X_tfidf = vectorizer.fit_transform(df["Processed_Text"])

print("TF-IDF Feature Names:\n", vectorizer.get_feature_names_out())
print("\nTF-IDF Matrix:\n", X_tfidf.toarray())
```

```
TF-IDF Feature Names:
 ['also' 'bad' 'blowing' 'call' 'deep' 'email' 'hate' 'language' 'learning'
 'love' 'machine' 'mind' 'natural' 'nlp' 'processing' 'product' 'spam'
 'technology']

TF-IDF Matrix:
 [[0.33333333 0.         0.         0.         0.33333333 0.
  0.         0.         0.66666667 0.33333333 0.33333333 0.
  0.         0.33333333 0.         0.         0.         0.        ]
 [0.         0.70710678 0.         0.         0.         0.
  0.         0.         0.         0.         0.         0.
  0.         0.         0.         0.70710678 0.         0.        ]
 [0.         0.         0.40824829 0.         0.         0.
  0.         0.40824829 0.         0.         0.         0.40824829
  0.40824829 0.         0.40824829 0.         0.         0.40824829]
 [0.         0.         0.         0.5        0.         0.5
  0.5        0.         0.         0.         0.         0.
  0.         0.         0.         0.5        0.         0.        ]]
```

Save Outputs

```
# Save processed dataset
df.to_csv("processed_output.csv", index=False)

# Save TF-IDF matrix
tfidf_df = pd.DataFrame(X_tfidf.toarray(), columns=vectorizer.get_feature_na
tfidf_df.to_csv("tfidf_output.csv", index=False)
```

```
print("Files Saved Successfully!")
```

```
Files Saved Successfully!
```

Download PDF

```
from google.colab import files
files.download("processed_output.csv")
files.download("tfidf_output.csv")
```